The Journal of Open Source Software

QuietZero: A Python Framework for Neural Architecture Search using Swarm Intelligence

Tashin Ahmed [1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 05 March 2026
Published: unpublished

Summary

Neural Architecture Search (NAS) has become an essential mechanism for automating the design of machine learning models. `QuietZero` is a comprehensive Python library designed to automatically discover optimal Neural Network (NN) architectures by leveraging swarm intelligence algorithms. Utilizing a seamless PyTorch backend (Paszke et al., 2019), the framework abstracts the complexities of distributed architecture evaluation and provides researchers with a unified API to access nine distinct nature-inspired optimization algorithms: Ant Colony Optimization (ACO) (Dorigo & Stützle, 2004), Particle Swarm Optimization (PSO) (Kennedy & Eberhart, 1995), Artificial Bee Colony (ABC) (Karaboga, 2005), Grey Wolf Optimizer (GWO) (Mirjalili et al., 2014), Whale Optimization Algorithm (WOA) (Mirjalili & Lewis, 2016), Cuckoo Search (CS) (Yang & Deb, 2009), Firefly Algorithm (FA) (Yang, 2008), Bat Algorithm (BA) (Yang, 2010), and Moth Flame Optimizer (MFO) (Mirjalili, 2015). `QuietZero` enables researchers to efficiently navigate discrete, high-dimensional search spaces to generate performant NN topologies.

State of the Field

NAS has emerged as a critical subfield of AutoML, aimed at automating the design of NN architectures. The field has evolved significantly since its introduction, with methods generally categorized into three main paradigms: reinforcement learning-based approaches, evolutionary algorithm-based approaches, and gradient-based differentiable methods.

Reinforcement learning-based NAS, pioneered by Zoph and Le (Zoph et al., 2018), treats architecture generation as a sequential decision process where a controller network learns to output architecture descriptions. While effective, these methods typically require substantial computational resources, often measured in thousands of GPU-days. Evolutionary approaches, such as those surveyed by Real et al. (Real et al., 2019), apply principles of natural selection to evolve populations of architectures, maintaining diversity through mutation and crossover operations.

Gradient-based methods, particularly Differentiable Architecture Search (DARTS), introduced the concept of continuous relaxation of the architecture space, enabling efficient optimization through standard gradient descent. However, these methods introduce their own challenges, including the need for differentiable operations and susceptibility to aggregation bias in discrete search spaces.

Swarm intelligence algorithms represent a distinct paradigm that has received comparatively less attention in the NAS literature. These nature-inspired metaheuristics including Ant Colony Optimization, Particle Swarm Optimization, and their many variants offer several theoretical advantages for architecture search. They natively handle discrete search spaces without requiring relaxation, maintain explicit exploration-exploitation balances through population-based search,

and provide mechanisms for escaping local optima through stochastic perturbations and collective intelligence behaviors.

Despite their potential, existing implementations of swarm based NAS remain fragmented. The original DeepSwarm (Pattio, 2019) demonstrated the viability of Ant Colony Optimization for neural architecture discovery but was limited to a single algorithm and lacked comprehensive benchmarking infrastructure. Other implementations typically focus on individual algorithms in isolation, making comparative research difficult. This fragmentation contrasts sharply with the broader NAS ecosystem, where frameworks like NNI (Neural Network Intelligence) and AutoKeras provide unified interfaces for multiple search strategies.

The swarm intelligence optimization field itself has expanded considerably, with numerous algorithms proposed over the past two decades. The Grey Wolf Optimizer (Mirjalili et al., 2014), Whale Optimization Algorithm (Mirjalili & Lewis, 2016), and Moth Flame Optimizer (Mirjalili, 2015) represent newer additions that have shown competitive performance on standard optimization benchmarks. However, their application to NAS remains largely unexplored, presenting an opportunity for systematic investigation.

Statement of Need and Research Impact

Designing effective NNs is notoriously resource-intensive, often relying on extensive human trial-and-error and domain expertise. While gradient-based NAS methods such as those by (Zoph et al., 2018) and (Real et al., 2019) are popular, heuristic-based swarm intelligence offers significant advantages for exploring non-differentiable search spaces and escaping local optima. Researchers in computational intelligence and evolutionary algorithms require robust, parallelized tools that integrate naturally with modern deep learning ecosystems to study these behaviors.

`QuietZero` addresses this need by providing a scalable, multi-GPU capable environment tailored for experimentation. While it builds upon the foundational concepts of DeepSwarm (Pattio, 2019), `QuietZero` substantially expands the methodological scope from a single algorithm to a comprehensive suite of nine optimizers. The software enables credible near-term significance by allowing researchers to conduct reproducible comparative benchmarks of different swarm behaviors on identical search spaces. Built-in TensorBoard integration allows for real-time tracking of architectural evolution, making it highly suitable for empirical research and educational demonstrations.

Design Thinking and Architecture

The primary abstraction is the `BaseOptimizer` class, which establishes a uniform interface (`generate_topology()`, `update()`, and `is_finished()`) for all nine swarm algorithms. This architectural trade-off ensures that domain researchers can implement novel swarm algorithms or tweak existing heuristics without needing to interact with the underlying deep learning engine.

Furthermore, because NAS experiments are computationally expensive and long-running, the framework's state management was designed with explicit serialization in mind. The `get_state()` and `set_state()` abstractions ensure that the entire swarm's progress can be reliably checkpointed and resumed across distributed worker nodes.

Software Design

`QuietZero` is architected around a modular, extensible design philosophy that separates concerns across three primary layers: the optimization layer, the evaluation layer, and the orchestration layer.

Optimization Layer

At the foundation lies the `BaseOptimizer` abstract base class, which defines a uniform interface that all swarm algorithms must implement. This interface consists of three core methods: `generate_topology()` returns candidate architectures along with associated state information for tracking, `update()` incorporates fitness evaluations back into the optimizer's internal state, and `is_finished()` determines search termination criteria. This abstraction enables algorithmic plug-and-play; researchers can implement new swarm algorithms by subclassing `BaseOptimizer` and implementing these three methods, without requiring knowledge of the underlying neural network training infrastructure.

Each optimizer maintains its own state representation appropriate to its algorithmic paradigm. The `AntColony` optimizer, for instance, manages a pheromone matrix over the discrete architecture search space, while the `ParticleSwarm` optimizer tracks position and velocity vectors for each particle in a continuous parameterization of the space. The `ArtificialBeeColony` optimizer maintains employed bee, onlooker bee, and scout bee populations with associated food source memories. Despite these internal differences, all optimizers expose the same external interface, enabling transparent substitution.

Evaluation Layer

The evaluation layer handles neural network instantiation, training, and validation. Built on PyTorch (Paszke et al., 2019), this layer accepts topology specifications from the optimization layer and constructs corresponding neural network models. The search space specification supports configurable layer types including convolutional layers with tunable filter counts, kernel sizes, and activation functions; pooling layers with configurable types and sizes; and dense layers with variable unit counts.

Training configuration is similarly flexible, supporting specification of learning rates, batch sizes, epoch counts, and optimizer types. The evaluation layer returns fitness metrics typically validation accuracy back to the optimization layer for state updates. This clean separation enables the optimization algorithms to remain agnostic to the details of model training, while the evaluation layer remains agnostic to which algorithm generated the architectures.

Orchestration Layer

The `DeepSwarm` class serves as the primary orchestrator, coordinating between optimizers and evaluators while managing the overall search lifecycle. It handles checkpoint serialization through `save_checkpoint()` and `load_checkpoint()` methods that capture both optimizer state and search progress, enabling long-running experiments to be paused and resumed across session boundaries.

The orchestration layer also provides optional TensorBoard integration for real time experiment monitoring. When enabled, metrics including per iteration best rewards, population diversity measures, and architecture complexity statistics are logged for visualization. This infrastructure proves particularly valuable for comparative algorithm research, where understanding search dynamics is as important as final performance.

Search Space Configuration

Architecture search spaces are defined declaratively through YAML configuration files or Python dictionaries. Each layer type specifies discrete choices for hyperparameters—for example, a convolutional layer might allow filter counts of [16, 32, 64, 128], kernel sizes of [3, 5, 7], and activations of ['relu', 'tanh']. This combinatorial specification defines the discrete search space that swarm algorithms navigate. The framework supports hierarchical architecture patterns, enabling specification of layer sequences with variable depths.

Empirical Evaluation

To demonstrate the effectiveness of the implemented swarm intelligence algorithms, we conducted benchmark experiments across three standard datasets: MNIST, Fashion-MNIST, and CIFAR-10. Figure 1 shows the accuracy comparison of different swarm algorithms on each dataset.

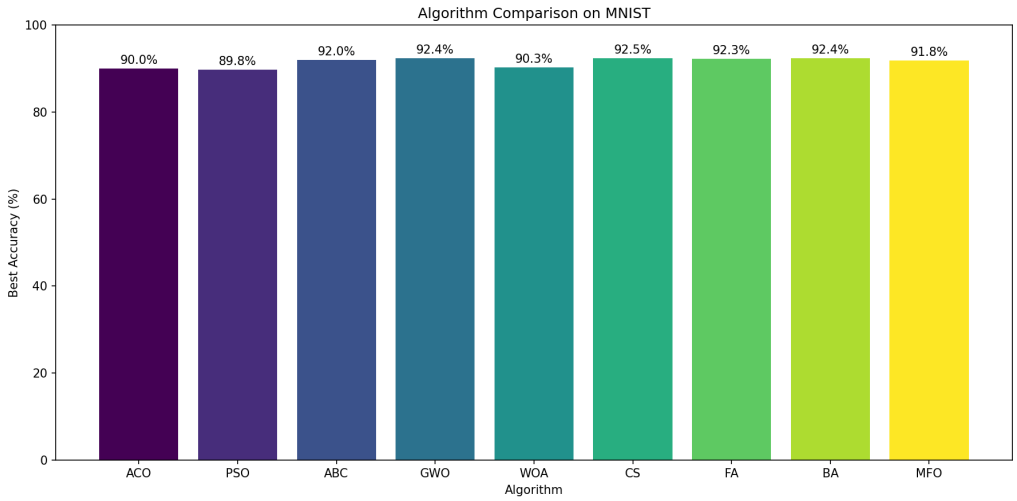

Figure 1a: MNIST - Algorithm accuracy comparison

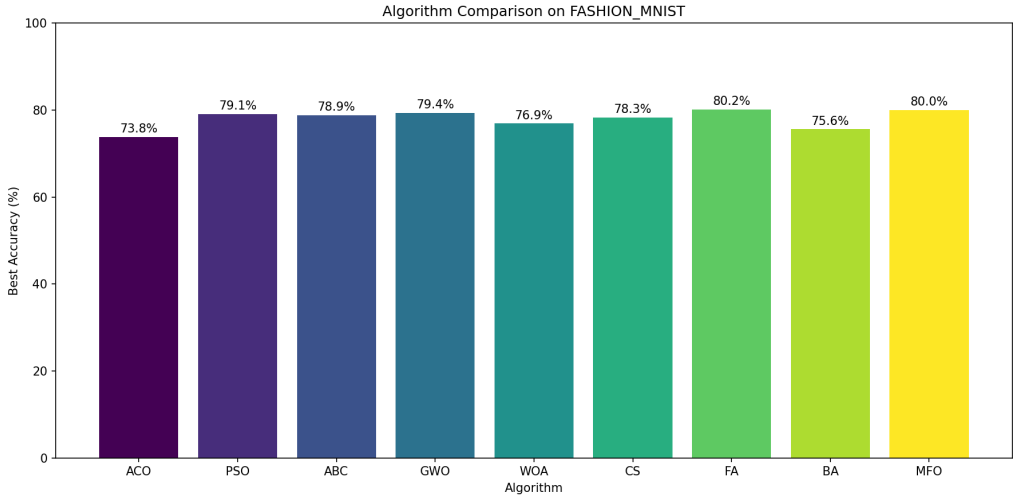

Figure 1b: Fashion-MNIST - Algorithm accuracy comparison

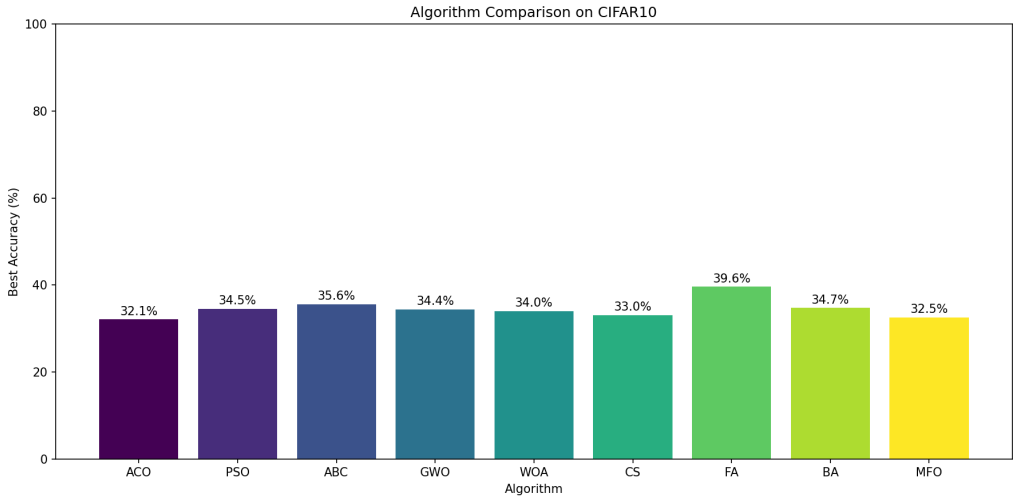

Figure 1c: CIFAR-10 - Algorithm accuracy comparison

The results demonstrate that all nine swarm algorithms are capable of discovering competitive NN architectures, validating the framework's utility for comparative algorithm research in the NAS domain.

Research Impact

`QuietZero` enables several categories of research contributions that were previously difficult or impossible with existing tools.

Comparative Algorithm Benchmarking

The framework's unified implementation of nine swarm algorithms on identical infrastructure enables, for the first time, rigorous head-to-head comparisons of these methods on neural architecture search tasks. Researchers can now investigate questions such as: Which swarm algorithms perform best on different dataset complexities? How do convergence rates compare across algorithms? What are the computational trade-offs between population size and iteration count for each method? Our empirical evaluation across MNIST, Fashion-MNIST, and CIFAR-10 provides initial answers to these questions while establishing baselines for future research.

Novel Algorithm Development

The extensible `BaseOptimizer` interface lowers the barrier to developing and evaluating new swarm intelligence algorithms for NAS. Researchers proposing algorithmic modifications such as adaptive parameter schedules, hybrid algorithms combining multiple swarm behaviors, or novel metaheuristics can implement their methods within the framework and immediately benchmark against the existing suite. This infrastructure accelerates the iteration cycle for algorithmic research.

NAS Methodology Research

Beyond algorithm comparison, `QuietZero` supports broader methodological investigations in neural architecture search. The discrete nature of swarm-based search contrasts with continuous gradient-based methods, raising questions about search space structure, the role of locality in architecture performance landscapes, and the relationship between exploration strategies and architecture quality. The framework's detailed logging and checkpointing infrastructure enables post-hoc analysis of search trajectories to address these questions.

Educational Applications

The framework's modular design and comprehensive documentation make it suitable for educational contexts. Students learning about swarm intelligence can observe algorithm behavior on a practical, consequential optimization problem rather than abstract benchmark functions. The visualization capabilities support intuition-building about how different swarm behaviors manifest in high-dimensional discrete spaces.

Reproducibility and Replicability

By providing a single, well-tested implementation of multiple algorithms, `QuietZero` addresses reproducibility challenges that plague metaheuristic research. Implementations of swarm algorithms in the literature often differ in subtle details that significantly impact performance. The framework's standardized implementations, combined with version-controlled releases, ensure that reported results can be reliably reproduced and extended by independent researchers.

Open-Source Practices

`QuietZero` was developed with a strong commitment to sustainable open-source software practices. The project provides a well-documented user-facing API and is distributed via PyPI with clearly versioned releases. To maintain a clean ecosystem, dependencies are thoughtfully managed through targeted installation groups (e.g., `quietzero[tensorboard]` and `quietzero[dev]`). The codebase is released under the permissive MIT license, incorporates comprehensive examples for standard datasets (MNIST, CIFAR-10), and maintains a transparent issue-tracking pathway on GitHub to foster ongoing community contributions and algorithmic extensions.

AI Usage Disclosure

During the preparation of this submission, private generative AI was utilized to assist in structuring the initial draft and formatting the Markdown text in accordance with JOSS guidelines. The author comprehensively reviewed, edited, and validated all AI-assisted outputs. The author maintains full responsibility for the core software architecture, the problem framing, all technical claims, and the accuracy of this manuscript.

Acknowledgements

`QuietZero` is an upgraded and expanded iteration inspired by the original DeepSwarm project created by Pattio.

References

Dorigo, M., & Stützle, T. (2004). *Ant colony optimization*. MIT Press.

Karaboga, D. (2005). *An idea based on honey bee swarm for numerical optimization* (No. TR06). Erciyes University, Engineering Faculty, Computer Engineering Department.

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 - International Conference on Neural Networks*, *4*, 1942–1948.

Mirjalili, S. (2015). Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm. *Knowledge-Based Systems*, *89*, 228–249.

Mirjalili, S., & Lewis, A. (2016). The whale optimization algorithm. *Advances in Engineering Software*, *95*, 51–67.

Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). Grey wolf optimizer. *Advances in Engineering Software*, *69*, 46–61.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*, 8024–8035. http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

Pattio. (2019). *DeepSwarm: Neural architecture search with ant colony optimization*. GitHub. https://github.com/Pattio/DeepSwarm

Real, E., Aggarwal, A., Huang, Y., & Le, Q. V. (2019). Regularized evolution for image classifier architecture search. *Proceedings of the AAAI Conference on Artificial Intelligence*, *33*, 4780–4789.

Yang, X.-S. (2008). *Nature-inspired metaheuristic algorithms*. Luniver Press.

Yang, X.-S. (2010). A new metaheuristic bat-inspired algorithm. In *Nature inspired cooperative strategies for optimization* (pp. 65–74). Springer.

Yang, X.-S., & Deb, S. (2009). Cuckoo search via lévy flights. *World Congress on Nature & Biologically Inspired Computing*, 210–214.

Zoph, B., Vasudevan, V., Shlens, J., & Le, Q. V. (2018). Learning transferable architectures for scalable image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 8697–8710.